Improving Pattern-Matching Algorithm Using Counting Set Automata

팀 오토마토

곽재혁, 서기석, 김진주

Professor : 한요섭

Assistant : 성시철

[Contents]

1 Background

1.1 Regular Expression

1.2 Pattern Matching Algorithm

2 Related Research

3 Proposed Research

4 Research Methodology

5 Research Process

6 Result and Conclusion

7 Limitation and Future Work

8 Team Member

9 Reference

1. Background

1.1 Regular Expression

정규표현식(Regular Expression, RegEx)은 문자열에서 특정 패턴을 찾거나 변형하는 데 사용되는 도구로, Stephen Cole Kleene가 제시한 정규 언어 이론[2]에서 기원되었다. 정규표현식은 다양한 메타문자와 연산자를 사용해 특정 패턴을 정의하는데, 이를 통해 텍스트에서 원하는 정보를 쉽게 추출하거나 변형할 수 있다. 이론적으로는 집합 이론과 형식 언어 이론에 기반을 두고 있으며, 실제로는 텍스트 데이터를 처리하는 데 중요한 역할을 한다. 정규표현식은 데이터 정제, 로그 파일 분석, 웹 크롤링, 자연어 처리(NLP) 같은 분야의 기반이 되는 표현 도구이다[3].

1.2 Pattern Matching Algorithm

정규표현식 매칭(RegEx-Matching)은 텍스트 데이터에서 특정 문자열이나 패턴을 찾아내는 알고리즘이다. 따라서 데이터 검색, 데이터 유효성 검사, 데이터 누수 탐지 등 문자열 searching이 요구되는 분야에 항상 사용된다. 또한, 자바와 파이썬 등 대부분의 프로그래밍 언어에는 parsing과 replacing, checking, 그리고 labeling과 같은 기본 함수가 내장되어있는데, 이처럼 배열이나 문자열과 같은 데이터 집합체에서 주어진 데이터를 특정할 때에도 역시 정규 표현식 매칭이 사용된다.

특정 패턴의 반복 횟수를 기억할 수 있는 정규표현식 매칭은 현재 프로그램에서 가장 많이 쓰이는 알고리즘이다. 그러나 횟수를 기억하기 위해 사용되는 기존 백트래킹 방식은 데이터 양이 많을수록 시간 복잡도가 급격히 증가한다는 문제점이 있다. 이로 인해 기존 정규 표현식 매칭 방식은 대규모 텍스트를 처리할 때 프로그램에 심각한 성능 저하를 초래한다[1].

2. Related Research

Counting Automata(CA)는 여러 카운터 변수를 사용하여 문자열 매칭을 수행하는 오토마타로, NFA와 비교했을 때 특정 패턴의 반복 횟수를 기억할 수 있다는 점에서 차별점이 있다. CA는 데이터 셋을 처리할 때 발생할 수 있는 모든 과정을 상태로 정리하여 저장한 super-configuration 개념을 도입하여 효율적인 매칭을 수행한다. 그러나 일반적인 CA는 super-configuration의 크기는 패턴 반복 횟수가 증가함에 따라 전체 상태 수보다 커질 수 있어 선형 시간 복잡도를 보장할 수 없다.

기존 연구에서는 CA의 매칭 알고리즘을 다루었으나, super-configuration의 크기를 제어하는 방법에 대한 연구는 잘 이루어지지 않았다. 이때, "Fast Matching of Regular Patterns with Synchronizing Counting"[1]에서는 Counting-Set Automata를 사용하여 반복 횟수에 관계없이 일정하게 빠른 매칭을 구현할 수 있는 가능성을 제시했다. 이 연구는 기존의 백트래킹 방식이 아닌 CSA 기반의 비결정적 오토마타 방식을 사용하여 효율성을 크게 향상시킬 수 있음을 보여준다. 따라서 해당 연구를 기반으로 실제 프로그램을 설계하여, 패턴 반복 횟수를 기억하는 정규 표현식 매칭이 기존보다 효율적으로 이루어질 수 있음을 보이고자 한다.

3. Proposed Research

본 연구의 목표는 C 언어를 이용하여 Counting-Set Automata (CSA)를 알고리즘으로 구현하고, 기존의 정규 표현식 매칭 알고리즘과 비교하여 성능을 평가하는 것이다. 세부적으로는 아래에 적힌 소목표로 나뉜다.

- 단순 오토마타 구현 및 transition 등 구성 개념 설계
- Counting Automata(CA) 및 Counting-Set Automata(CSA) 구현

- 기존의 백트래킹 방식과 CSA 방식의 성능 비교 및 분석
- CSA의 성능 향상을 위한 알고리즘 최적화 및 개선 방안 제안

4. Research Methodology

1. 오토마타 구현 (C 언어 사용):
    - 기본적인 오토마타(Non-Deterministic Finite Automata) 구현
    - 단순한 정규 표현식 매칭을 구현하여 오토마타 작동 원리 이해 및 기본 성능 테스트
2. Counting Automata 및 Counting-Set Automata 구현:
    - Holik 외 연구 논문을 기반으로 CSA의 이론적 구성 요소 구현
    - 상태 집합을 관리하는 방법 고안 후 카운터 변수를 포함한 오토마타 구조 설계
    - CSA의 Transition, Guard, Action 정의 및 구현
3. 성능 평가:
    - 구현된 CSA 알고리즘을 다양한 입력 문자열에 대해 테스트
    - 기존의 백트래킹 방식과 비교하여 시간 복잡도 및 성능 측정
4. 알고리즘 개선:
    - 실험 결과를 바탕으로 CSA의 성능을 향상시키기 위한 최적화 방법 연구
    - 기존 CSA의 한계를 극복할 수 있는 추가 기능 설계 및 구현

5. Research Process

1. CSA

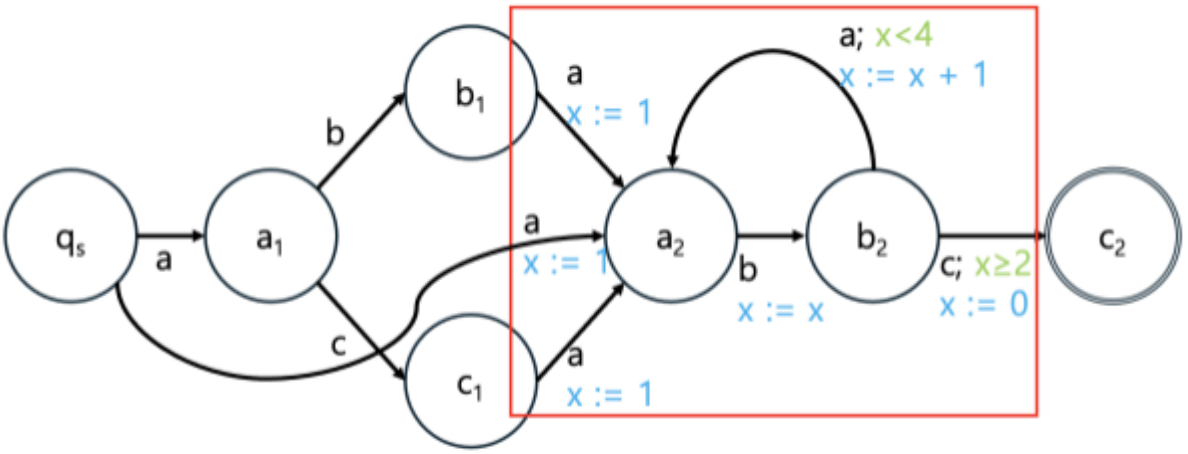
1.1 Case Study

정규표현식의 일반적인 사용에서는 백트래킹 방식이 크게 문제가 되지 않는다. 하지만 정규표현식을 이용한 서비스 거부 공격 (ReDos) 사례를 살펴보면 주로 정규표현식의 중첩된 수량자가 문제가 되었다. Stack

Overflow에서 문장 끝의 공백을 검사하기 위해 사용된 “\s+$” 패턴의 경우 “^.*\s+$”와 동치이며, 공백 개수에 대해 이차 함수적인 시간이 소모되어 34분가량 서비스가 중단된 사례가 있었다. [4] 개발자가 직접 작성한 정규표현식이 아닌, Express.js에서 자동 생성된 라우팅 패턴 “^(([^\/?#]{1,5})+)+$” 등에서도 중첩된 수량자를 확인할 수 있어 ReDos 공격에 악용될 수 있다. 이메일 주소 검증은 정규표현식 사용의 대표적인 사례지만, RegexLib에 등록된 예시(ReGexLib,id=1757 (email validation)) 역시 수량자 중첩 문제를 포함하여 이메일 길이에 대해 최대 지수시간의 시간이 소모될 수 있다. 따라서 입력 길이에 비례하는 선형 시간 내에 패턴 매칭이 수행되도록 설계할 필요가 있다.

1.2 Counting Automata

수량자가 포함된 경우 오토마타 더불어 현재 반복이 몇 번째 진행중인지 저장하는 counter 변수가 필요하다. 정규표현식 “(a(b|c))?(ab){2,4}c”의 경우 (anchor는 생략하였다) 아래와 같은 오토마톤이 구성된다.



정규표현식 “(a(b|c))?(ab){2,4}c”에 해당하는 오토마톤. 빨간 상자 안쪽이 수량자에 해당하는 부분이다. 파란색 부분은 counter (x)에 적용하는 연산을 의미하며 초록색 부분은 guard를 의미한다. q_s가 시작 state이며, final state는 {c_2}이다.

counter에는 반복이 시작될 때 active하는 :=1 연산, 반복을 마친 후 inactive하는 :=0 연산이 필요하며, 반복횟수를 유지하거나 (:=x) 다음 반복을 시작하는 increment(:=x+1) 연산이 적용될 수 있다. counter 값에 따라 transition의 가능 여부가 제한되고, 이 제한을 guard라고 한다.

비결정적 유한 오토마타(NFA)이므로 현재 가능한 state 뿐만 아니라 counter 값으로 가질 수 있는 모든 경우를 저장하고 있어야 한다.

counter는 최대 반복횟수만큼의 값을 가질 수 있으므로, 각각에 연산을 적용할 때 최대 반복횟수에 대해 선형 시간에 작동한다. 최대 반복횟수는 정규표현식에 나타나는 수량자의 값에 따라 결정되어 정규표현식 길이에 지수함수만큼 비례한다. 따라서 여러 counter 값에 대해 한 번에 연산을 적용하여 최대 반복횟수와 무관한 시간에 작동하도록 Counting Set Automata[1]를 적용한다.

1.3 Counting Set Automata

Counting Set Automata에서는 counter값을 offset-list를 사용하여 저장한다. State q의 offset o_q와 list $l_q = (m_1, \dots, m_k)$는 list의 각 값에 offset값을 더한 $\{o_q + m_1, \dots, o_q + m_k\}$를 나타내며, 이 때 list는 오름차순으로 저장되어 있으며 최소값과 최대값을 가리키는 포인터를 갖고있다.

1.3.1 값의 유지 및 증가 연산

State를 이동할 때 반복횟수를 유지하는 경우, State가 offset-list를 가리키는 포인터를 교환하고, increment 연산의 경우에는 offset만 1을 더해주면 되기에 상수 시간에 연산을 처리할 수 있다.

1.3.2 전체 리스트를 비우는 연산

Active, Inactive 연산의 경우 기존 list를 비우고 새로운 state에 0 또는 1을 삽입하는 과정에서 리스트를 비우는 연산은 리스트 길이에

선형시간이 걸리지만, offset-list에 삽입되는 값이 0 또는 1이라는 점을 이용하면 삽입 연산에 의해 상수 시간으로 amortized된다.

1.3.3 offset-list의 합집합 연산

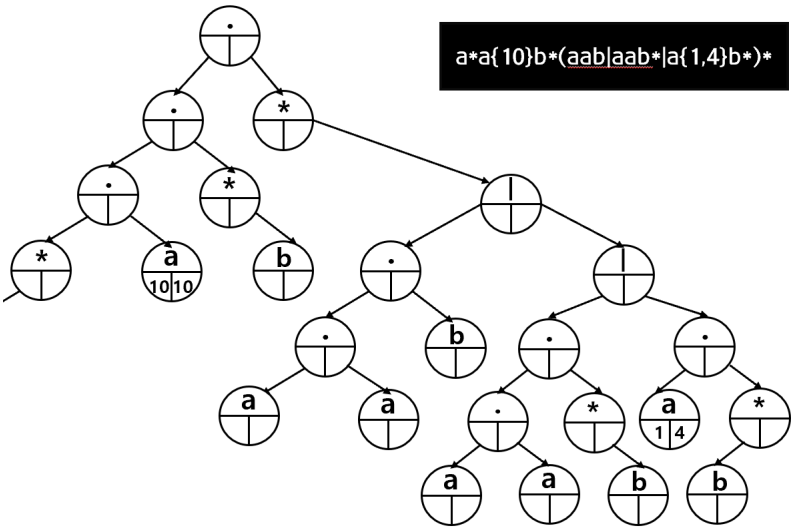
여러 state에서 하나의 state로 transition이 발생하여 offset-list의 합집합 연산이 필요할 수 있는데, (위의 예시에서 aba의 경우를 생각한다.) increment 연산이 하나의 symbol에 대해 최대 한 번만 일어난다는 점을 이용하면 increment 연산에 의해 상수시간으로 amortized된다.

1.3.4 비교 연산

최소값 혹은 최대값만 비교하여 guard 조건을 만족하는지 확인할 수 있으므로 이 또한 상수시간에 검사 가능하다.

2. Optimizing RegEx

1.1 Case Study



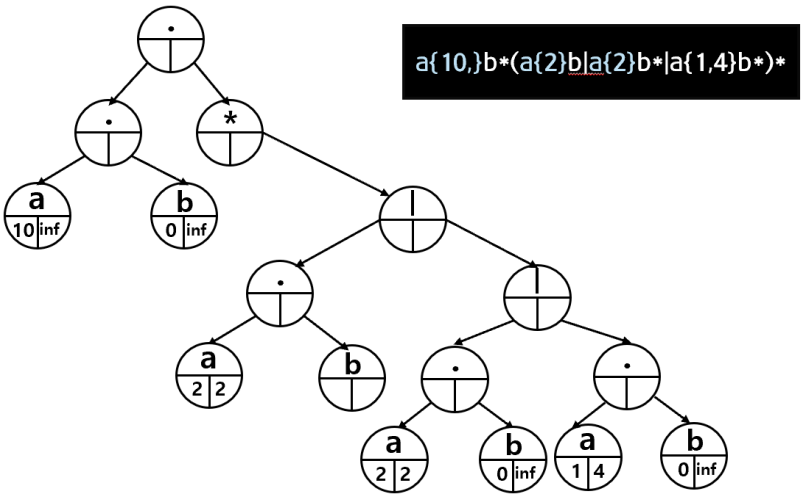
정규표현식 “`a*a{100}`”에 대해, a가 101번 반복된 문자열이 들어온다면 오토마타는 문자열을 한 글자씩 받아들이기 때문에 몇 번째 a까지 `a*`에 매칭하고 `a{100}`으로 넘어가야할지 결정하지 못한다. 따라서 a를 101개부터 1개까지 매칭하는 경우의 수를 전부 살펴보는 비효율적인 연산이 발생하며, 이는 그 어떤 RegEx에 대해 매칭 과정에서 linear한 시간복잡도를

보장하는데 문제를 야기한다. 따라서 RegEx를 parsing하는 과정에서 “a*a{100}”을 “a{100,}”으로 변환하는 과정을 거쳐 이후 parse-tree를 순회할 때 경우의 수를 최대한 줄이고자 한다.

이처럼 수량자를 이용하여 RegEx를 축약시키는 과정의 경우, 오토마타의 관점에서 state 수가 줄어들 순 있지만 configuration의 크기는 바뀌지 않는다. 그러나 CSA를 사용하면 한 state의 여러 configuration에 대한 연산을 한 번에 처리할 수 있으므로 매칭 시간 감소를 기대할 수 있다. 또한 DFA 변환을 유도하는 타 매칭 방법론과 함께 사용하는 경우 직접적인 시간복잡도 감소로 이어진다. 앞에서 언급한 Stack overflow의 ReDos 공격 사례 “^.*\s+$”에서, 공백 한 칸은 .과 \s 모두에 매칭되는 경우이므로 이 방법론을 확장하여 적용할 수 있다.

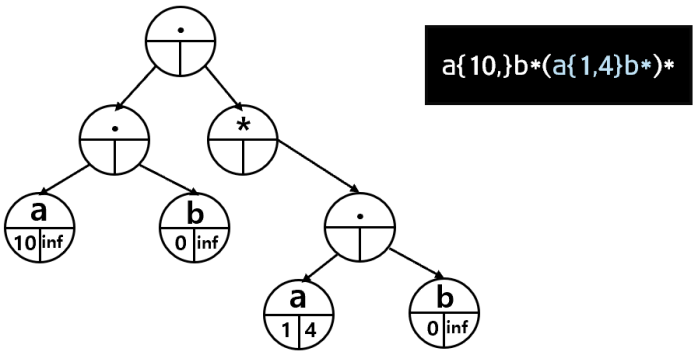
1.2 Symbol Optimization

순차적으로 등장하는 동일 심볼 노드들에 대한 최적화이다. 기본적인 parsing 과정이 끝난 후 parse-tree를 단일순회(깊이 우선 탐색)하며 직전 노드와 현재 노드, 그리고 kleene star 여부를 비교하여 symbol의 일치 여부를 살피고, 범위에 따라 통합 가능한 범위를 합쳐 단일 심볼에 대한 특정 범위로 축약한다.



1.3 Pattern Optimization

다중 심볼 노드에 대해, 만약 parse-tree 내 동일한 심볼과 순서로 구성된 sub-tree가 존재한다면 이는 동일 패턴 노드로 이에 대한 최적화가 이루어질 수 있다. Symbol Optimization이 끝난 후 parse-tree를 이중순회하며 자식 노드를 가진 심볼 노드들 사이의 sub-tree 패턴 비교를 진행한다. 만약 sub-tree가 동일한 심볼과 순서로 순서로 이루어져 있다면, 각 단일 노드의 비교분석을 통해 범위의 통합 여부를 검사한다. 만약 모든 단일 노드의 범위가 통합 가능하다면 양쪽의 sub-tree를 하나의 sub-tree로 축약시켜준다.



6. Result and Conclusion

6.1 텍스트 길이의 증가에 따른 방법론별 매칭 step 수 증가 추세

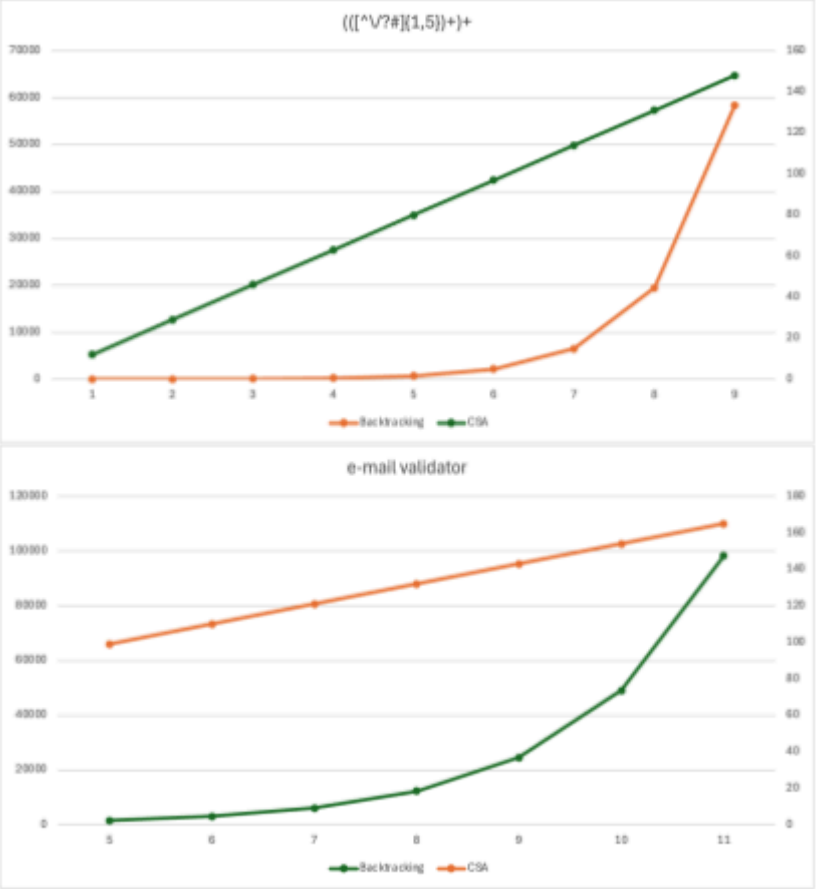
수량자 {m, n}이 포함된 정규표현식들과, ReDos 공격에 사용될 수 있는 (catastrophic backtracking이 일어날 수 있는) 인풋 형식에 대해 step수를 계산하였다. Backtracking 엔진은 regex101의 Python 엔진을 사용하였으며, CSA 엔진은 직접 구현하여 포인터 swap, list에 삽입 및 삭제되는 원소 수를 계산하였다. step 계산방식에 차이가 있으므로 절대적인 값이 아닌 증가 추세에 주목해야한다.

대표적인 사례로 Section 5.1에서 소개한 정규표현식 “`^(([^\/?#]{1,5})+)+$`” 와 이메일 검증 정규표현식을 선정하였으며, 이에

대한 step 수 증가 경향을 나타낸 그래프를 제시하였다. RegEx 내부 반복 횟수가 증가함에 따라 backtracking 방식으로 매칭을 진행한 기존 방법론은 비선형적인 step 증가율을 보이는 반면, CSA를 사용한 방식은 선형적인 step 증가율을 보인다.

6.2 반복 횟수에 증가에 대한 방법론별 매칭 step 수 증가 추세

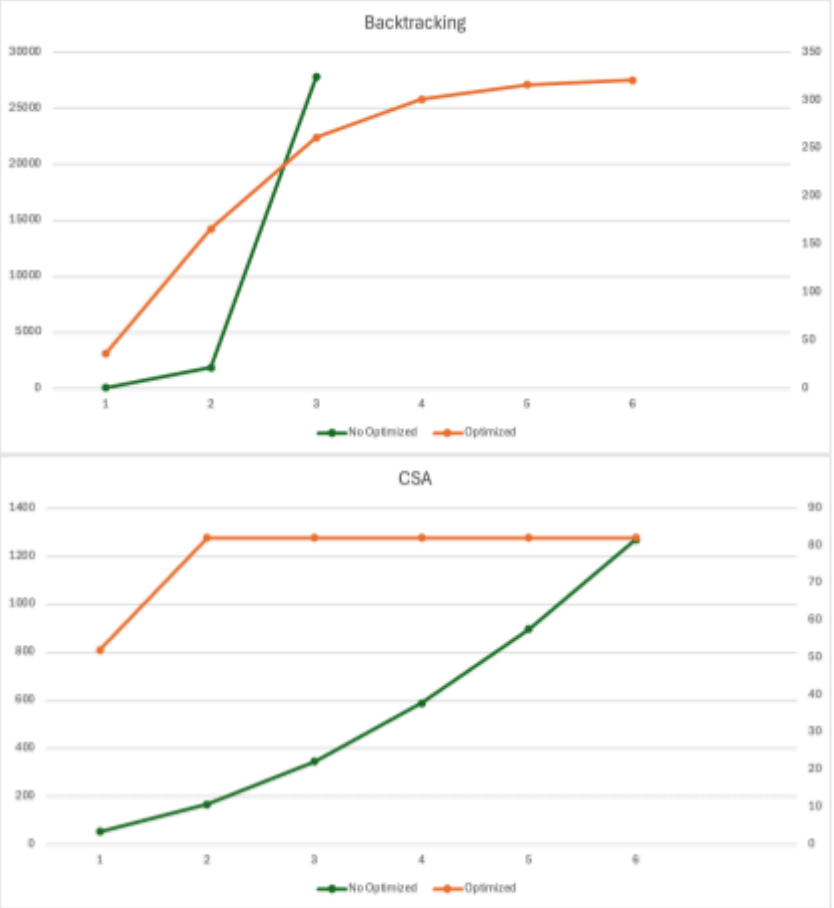
`{m,n}` 형태의 수량자에 대해 최대 반복 횟수 변화에 따른 step 수를 분석한 결과, 정규표현식의 형태에 따라 다양한 증가 경향이 관찰되었으나 CSA 방법론은 최대 반복



Section 5에서 언급한 정규표현식 대해 악의적 인풋 텍스트가 주어질 때의 step수 증가 추세이다. Backtracking 방식은 지수시간으로 증가하지만 CSA 방식은 선형 시간으로 증가함을 볼 수 있다.

횟수의 변화와 무관하게 일정한 증가 추세가 나타났으며, Section 5.2에서 언급한 기법이 효과적으로 작용함을 확인할 수 있었다.

대표적인 예시로 고전적인 ReDoS 공격 패턴인 “(a+)+$”을 변형하여 “^(a?a?a?)+$”와 Section 5.2에서 언급한 방법론을 적용한 “^(a{,3})+$”을 선정하였으며, 이에 대한 step 수 증가 경향을 나타낸 그래프를 제시하였다. 첫 번째 정규표현식에 대해 CSA 방법은 정규표현식 길이에 대해 이차 함수적인 시간 복잡도 증가 추세를 보이며, 오토마타 크기 측면에서는 선형적인 증가 양상을 나타낸다. 그러나 두 번째 정규표현식의 경우에는 반복 횟수에 대해 비례하지 않고 상수 시간으로 수렴한다. 이 정규표현식 예시에서는 백트래킹 방법에서도 상수 시간으로 수렴하는 것처럼 보이나, 인풋 텍스트에 여전히 취약한 모습을 보여 수렴속도가 매우 늦어져, CSA와 함께 사용했을 때 효과적인 방법론임을 확인할 수 있다.



Section 5.2에 언급한 방법론을 backtracking과 CSA엔진에 적용 전후를 비교한 그래프이다. “(a?a?a?)+”와 같은 정규표현식은 “(a{,3})”+로 축약되며, 두 엔진 모두 상수 시간으로 수렴하는 모습을 보이나, backtracking 엔진의 경우 input text 길이에 여전히 취약하여 수렴 속도가 매우 늦어지게 된다.

6.3 결론

본 연구에서 이론적으로만 증명되고 실제 구현이 이루어지지 않은 선제 연구를 실현하는 과정 중에 CSA의 문제점을 발견할 수 있었다. 문제점을 개선하기 위한 고민을 통해 기존 논문과 다른 방향의 해결책을 고안할 수 있었으며, 이에 대한 연구를 병행하여 기대한 알고리즘보다 더 안정적이고 빠른 알고리즘을 제작할 수 있었다.

CSA 구현 및 결과 분석을 통해, 오토마타에서 counting variable을 이용할 경우 기존의 백트래킹 방식보다 대규모 데이터에 대해 효율적으로 패턴 매칭을 수행할 수 있음을 실제 데이터에 기반하여 증명할 수 있었다.

결과에 근거하여, 거대한 문자열에서 특정 문자열을 찾거나 비교하는 데 있어 선형적인 시간복잡도에 가깝게 실행시간을 단축시킬 수 있을 것으로 예상되며, 이는 대다수 프로그램의 효율성을 크게 증가시킬 것이다.

6.4 데이터

Section 6에서 제시된 CSA 엔진과 step수 계산 결과 등 세부 사항은 다음 링크에서 확인할 수 있다.
(https://drive.google.com/drive/folders/1eYQBLRydc68WUI4VRdq0e2zUrhi4OKAT?usp=sharing)

7. Limitations and Future Work

한 state에서 여러 state로의 increment 또는 카운터 유지가 가능할 때 list의 복제가 일어나고, 복제 연산은 상수 시간에 처리할 수 없다. 이 경우 여러 state가 하나의 list를 갖게 하는 synchronizing 기법을 고려할 수 있다. 이 때 여러 state의 offset은 최대 1까지 차이가 나는 경우에만 offset-list를 관리할 수 있다는 한계가 있다. 또한 “((*expression*){m,n}(*expression*)){m′,n′}” 과 같이 {m,n}의 수량자가 중첩된 경우에도 synchronizing 기법으로 처리할 수 없으며, 복제

연산이 없더라도 중첩 깊이에 대한 이차함수 시간에 처리된다. [5]에 의하면 snort와 Regexlib에서 수집한 실제 정규표현식 중 각 counter의 최대 반복 횟수의 합이 20 이상인 정규표현식은 5751개가 있고, 오직 0.4%에 해당하는 24개만이 synchronizing 기법으로도 상수 시간에 처리할 수 없다. 그러나 극소수의 패턴이라 하더라도 ReDos 공격의 잠재적 요소가 될 수 있으므로, synchronizing 기법으로도 처리할 수 없는 형태의 정규표현식 매칭에 대한 추가 연구가 필요하다.

8. Team Members

- 곽재혁(팀장)
  - Glushkov Construction Algorithm을 이용하여 정규표현식을 오토마타로 변환하는 알고리즘 구현
  - Counter Automata을 기반으로 transition을 register에 저장하여 Counter Set Automata 구현
- 서기석
  - 결과 분석용 RegEx 스크래핑 및 데이터 분류 작업
- 김진주
  - RegEx Parsing 알고리즘 구현 및 Parse-Tree 구조 설계
  - 팀 자체 연구 기반 RegEx 최적화를 위한 중복 심볼 축약 알고리즘 구현 및 Parse-Tree 패턴 축약 알고리즘 구현
- 한요섭(담당 교수)
- 성시철(담당 연구원)

9. References

[1] Lukáš Holík, Juraj Síč, Lenka Turoňová, and Tomáš Vojnar. "Fast Matching of Regular Patterns with Synchronizing Counting." 2023.

[2] Kleene, S. C. (1956). "Representation of Events in Nerve Nets and Automata," *Automata Studies* (pp. 3–41). Princeton University Press.

[3] Friedl, J. E. F. (2006). “*Mastering Regular Expressions*” (3rd ed.) O'Reilly Media.

[4] 성시설, 한요섭. “ReDoS 발생 원인 및 기존 해결 방법의 한계점 분석” 2021.

[5] Turoˇnov´a, L., Hol´ık, L., Leng´al, O., Saarikivi, O., Veanes, M., Vojnar, T."Regex matching with counting-set automata". Proc. ACM Program. Lang. 4(OOPSLA) (2020) 218:1–218:30

[6] Henrik Björklund, Wim Martens, Thomas Timm. “Efficient Incremental Evaluation of Succinct Regular Expressions” 2015.

[6] Glushkov, V.M.: The abstract theory of automata. Russian Math. Surveys 16 (1961) 1–53

[7] Turoˇnov´a, L., Hol´ık, L., Leng´al, O., Veanes, M., Vojnar, T.: Counting in regexes considered harmful (2022)

[8] Smith, R., Estan, C., Jha, S., Siahaan, I.: Fast signature matching using extended finite automaton (XFA). In: ICISS’08. Volume 5352 of LNCS., Springer (2008) 158–172

[9] Gelade, W., Gyssens, M., Martens, W.: Regular expressions with counting: Weak versus strong determinism. In: Mathematical Foundations of Computer Science 2009, Berlin, Heidelberg, Springer Berlin Heidelberg (2009) 369–381

[10] Hovland, D.: The membership problem for regular expressions with unordered concatenation and numerical constraints. In: Language and Automata Theory and Applications, Berlin, Heidelberg, Springer Berlin Heidelberg (2012) 313–324